Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

Información general

Fecha de creación	29 – 09 - 2025	**Dirección de operaciones**

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Desarrollo de Interfaces Gráficas con Orientación a Objetos

Introducción

Imagina que has desarrollado un motor increíblemente potente y eficiente, una verdadera obra de ingeniería. Sin embargo, para que alguien pueda usarlo, debe conectar cables manualmente, verter combustible con un embudo y accionar palancas en una secuencia precisa. ¿Sería útil en el día a día? Probablemente no. Le faltaría un chasis, un volante, pedales y un tablero de instrumentos; en resumen, una interfaz que permita a un conductor interactuar con esa potencia de forma simple y segura. En el desarrollo de software, ocurre exactamente lo mismo. Un programa, por más brillante que sea su lógica interna, necesita una "cara" amigable y funcional para ser verdaderamente útil y accesible: una **Interfaz Gráfica de Usuario (GUI)**.

Este documento te guiará en el fascinante proceso de construir esa "cara" para tus aplicaciones. Daremos un paso fundamental más allá de las aplicaciones de consola, donde la interacción se limita a texto secuencial, para adentrarnos en el mundo de las ventanas, los botones, los menús y los formularios interactivos. Descubrirás que los principios de la Programación Orientada a Objetos (POO) —encapsulamiento, herencia y polimorfismo— no son solo conceptos teóricos, sino herramientas increíblemente poderosas que nos permiten construir interfaces complejas de manera ordenada, modular y reutilizable. Cada botón, cada ventana, cada panel, será un objeto con su propio estado y comportamiento, dialogando con otros objetos para crear una experiencia de usuario fluida.

La relevancia de aprender a desarrollar interfaces gráficas es innegable en el mercado laboral actual. Desde el software de punto de venta en una tienda en Tocancipá, hasta complejos sistemas de gestión empresarial o herramientas de software especializado, la capacidad de crear una interacción visual efectiva es una habilidad esencial para cualquier desarrollador. No se trata solo de hacer que los programas "se vean bonitos", sino de construir puentes lógicos entre la complejidad del código y la necesidad de simplicidad del usuario final.
Lo que aprenderás aquí no solo sirve para aprobar una asignatura, sino para dar vida a tus ideas. **Aprenderás a construir aplicaciones que no solo funcionan, sino que también dialogan con el usuario, transformando código en experiencias interactivas.**

Objetivos

El propósito fundamental de este documento es dotar al estudiante de las competencias conceptuales y técnicas necesarias para diseñar, desarrollar e implementar interfaces gráficas de usuario funcionales y robustas en el lenguaje de programación Java, aplicando de manera coherente los paradigmas de la Programación Orientada a Objetos.

Al finalizar el estudio de este material, el estudiante estará en capacidad de:

- **Analizar** los requerimientos de una aplicación para proponer un diseño de interfaz de usuario que sea intuitivo y resuelva las necesidades de interacción del usuario final.
- **Diferenciar** las arquitecturas y componentes de las principales librerías gráficas de Java (Swing y JavaFX), seleccionando la más adecuada según las especificaciones del proyecto.
- **Construir** ventanas interactivas y visualmente organizadas, utilizando contenedores, componentes y administradores de diseño (Layout Managers) para crear estructuras adaptables.
- **Implementar** el modelo de delegación de eventos para capturar y responder a las acciones del usuario (clics, pulsaciones de teclas, etc.), conectando los elementos visuales con la lógica subyacente del programa.
- **Vincular** la capa de presentación (la Vista) con la lógica de negocio (el Modelo), aplicando patrones de diseño como el MVC (Modelo-Vista-Controlador) para garantizar un código desacoplado, mantenible y escalable.

En esencia, este módulo busca responder a la pregunta "¿Para qué aprendo esto?" demostrando cómo la POO se materializa en productos de software tangibles con los que las personas pueden interactuar de forma directa y efectiva.

Fundamentos teóricos

En el mundo del software, la primera impresión es fundamental. Un programa puede tener la lógica más robusta y el algoritmo más eficiente, pero si el usuario no puede interactuar con él de manera intuitiva y agradable, su valor se ve drásticamente reducido. Aquí es donde entran en juego las Interfaces Gráficas de Usuario (GUI). Este capítulo sienta las bases teóricas que te permitirán comprender cómo la Programación Orientada a Objetos (POO) no solo facilita, sino que potencia la creación de estas ventanas interactivas. Pasaremos de un modelo de ejecución secuencial, donde el programa tiene el control total, a un universo dinámico dirigido por las acciones del usuario.

1.1. Programación Dirigida por Eventos: El Usuario Toma el Control

La programación tradicional, como la que se utiliza en aplicaciones de consola, sigue un flujo predecible y lineal. El programa pide un dato, el usuario lo ingresa; el programa procesa, muestra un resultado y termina. En cambio, la programación de interfaces gráficas opera bajo un paradigma completamente diferente: la **Programación Dirigida por Eventos** (Event-Driven Programming).

En este modelo, el programa se encuentra en un estado de espera activa, listo para reaccionar a una multitud de posibles acciones del usuario. Estas acciones, conocidas como **eventos**, pueden ser un clic de ratón, la pulsación de una tecla, el movimiento del puntero o el cambio de tamaño de una ventana. El flujo del programa ya no es dictado por el programador, sino por el usuario. Como analogía, piensa en el personal de un restaurante de comida rápida: no preparan todos los platos en un orden fijo al inicio del día; en su lugar, esperan y **reaccionan** a los pedidos (eventos) que hacen los clientes a medida que llegan (Joyanes Aguilar, 2020).

1.2. La Anatomía de una GUI: Componentes, Contenedores y Layouts

Para construir una interfaz gráfica, trabajamos con bloques de construcción visuales. En la POO, cada uno de estos bloques es, naturalmente, un objeto con sus propias propiedades (atributos) y comportamientos (métodos). Estos elementos se organizan en tres categorías principales:

1. **Componentes (o Widgets):** Son los elementos interactivos individuales con los que el usuario interactúa directamente. Son los "átomos" de nuestra interfaz. Ejemplos claros son los botones (JButton), las etiquetas de texto (JLabel), los campos de entrada (JTextField) y las casillas de verificación (JCheckBox).

2. **Contenedores:** Son componentes especiales cuya función principal es albergar y organizar a otros componentes (incluidos otros contenedores). Actúan como los "marcos" o "cajas" donde colocamos nuestros elementos. El contenedor principal y más fundamental es la ventana (JFrame), pero también existen paneles (JPanel) que nos ayudan a agrupar componentes dentro de una ventana.

3. **Layout Managers (Administradores de Diseño):** Son objetos invisibles pero cruciales que definen la política de posicionamiento y dimensionamiento de los componentes dentro de un contenedor. En lugar de especificar coordenadas exactas (ej: "coloca este botón en la posición x=50, y=100"), lo cual es frágil y poco adaptable, un Layout Manager lo hace automáticamente. Por ejemplo, FlowLayout los coloca uno tras otro, mientras que BorderLayout los organiza en cinco regiones: norte, sur, este, oeste y centro. Usar Layout Managers garantiza que nuestras interfaces se vean bien en diferentes tamaños de pantalla y resoluciones, una tarea clave para un tecnólogo en desarrollo de software (Deitel & Deitel, 2012).

1.3. El Trío Dinámico: Fuente del Evento, Objeto Evento y Listener (Oyente)

El mecanismo que conecta las acciones del usuario con la respuesta de nuestro código se basa en una delegación de responsabilidades entre tres actores clave. Comprender esta relación es fundamental para el desarrollo de cualquier GUI.

- **Fuente del Evento (Event Source):** Es el componente de la GUI sobre el cual el usuario realiza la acción. Por ejemplo, un JButton es la fuente de un evento de clic.
- **Objeto Evento (Event Object):** Cuando la acción ocurre, la fuente crea un objeto que encapsula toda la información relevante sobre el evento. Para un clic, este objeto (ej: ActionEvent) contiene detalles como qué componente se presionó y cuándo.
- **Listener u Oyente (Event Listener):** Es un objeto que está "a la escucha", esperando que ocurra un evento en una fuente específica. Para que esto funcione, debemos registrar explícitamente el Listener en la Fuente. El Listener contiene el método que se ejecutará como respuesta al evento.

Este modelo se conoce como el **Modelo de Delegación de Eventos**. La fuente delega la tarea de manejar el evento al oyente.

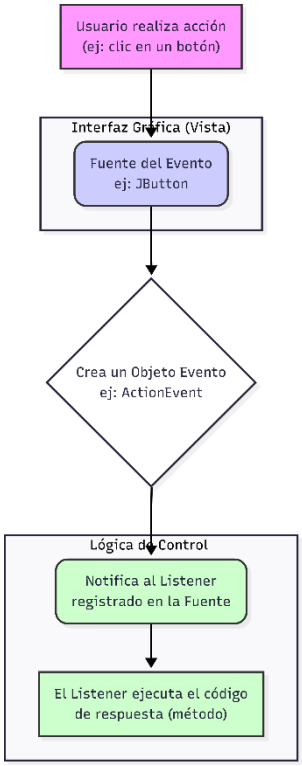

Figura 1 *Diagrama del Modelo de Delegación de Eventos*

Diagrama de flujo que muestra cómo una acción del usuario en un componente fuente genera un objeto evento que es enviado a un objeto listener para ejecutar una respuesta.

1.4. Librerías Gráficas en Java: De Swing a JavaFX

En el ecosistema de Java, existen principalmente dos bibliotecas para el desarrollo de GUIs de escritorio:

- **Swing:** Fue durante muchos años la librería estándar. Es robusta, multiplataforma y muy completa. Los nombres de sus clases suelen empezar con una "J" (e.g., JFrame, JButton). Una de sus ventajas es que los componentes son "ligeros", lo que significa que son dibujados enteramente por Java, dándoles una apariencia consistente en cualquier sistema operativo (Stallings, 2005).
- **JavaFX:** Es la evolución moderna de Swing. Fue diseñada para sucederla, ofreciendo un conjunto de herramientas más rico, soporte para propiedades, binding, efectos visuales y la capacidad de personalizar estilos de una manera mucho más sencilla y potente a través de CSS (Cascading Style Sheets). A nivel de un tecnólogo, es importante conocer ambas: Swing por su legado y la enorme cantidad de software existente, y JavaFX por ser la apuesta de futuro y ofrecer capacidades más avanzadas (Schildt, 2019).

1.5. Patrones de Diseño para GUI: Modelo-Vista-Controlador (MVC)

A medida que una aplicación crece, mezclar el código de la interfaz gráfica (la "vista") con la lógica de negocio (el "modelo") se convierte en una receta para el desastre. El mantenimiento se vuelve una pesadilla y la reutilización de código es casi imposible. Para resolver esto, se utiliza el patrón de diseño **Modelo-Vista-Controlador (MVC)**.
Este patrón propone separar la aplicación en tres componentes interconectados:

1. **Modelo (Model):** Representa los datos y la lógica de negocio de la aplicación. Es independiente de la interfaz de usuario. Por ejemplo, en una aplicación bancaria, el Modelo contendría las clases CuentaBancaria con métodos como depositar() y retirar().
2. **Vista (View):** Es la representación visual de los datos del Modelo. En nuestro caso, son las ventanas, botones y componentes de Swing o JavaFX. La Vista es responsable de mostrar la información al usuario.
3. **Controlador (Controller):** Actúa como intermediario entre el Modelo y la Vista. Recibe los eventos del usuario desde la Vista (como clics de botón) y los traduce en acciones a ejecutar sobre el Modelo. Una vez que el Modelo cambia, el Controlador puede notificar a la Vista para que se actualice (Sommerville, 2011).

Esta separación es clave para un desarrollo profesional, ya que permite que distintos desarrolladores trabajen en paralelo (unos en la Vista, otros en el Modelo) y facilita enormemente las pruebas y la evolución del software.

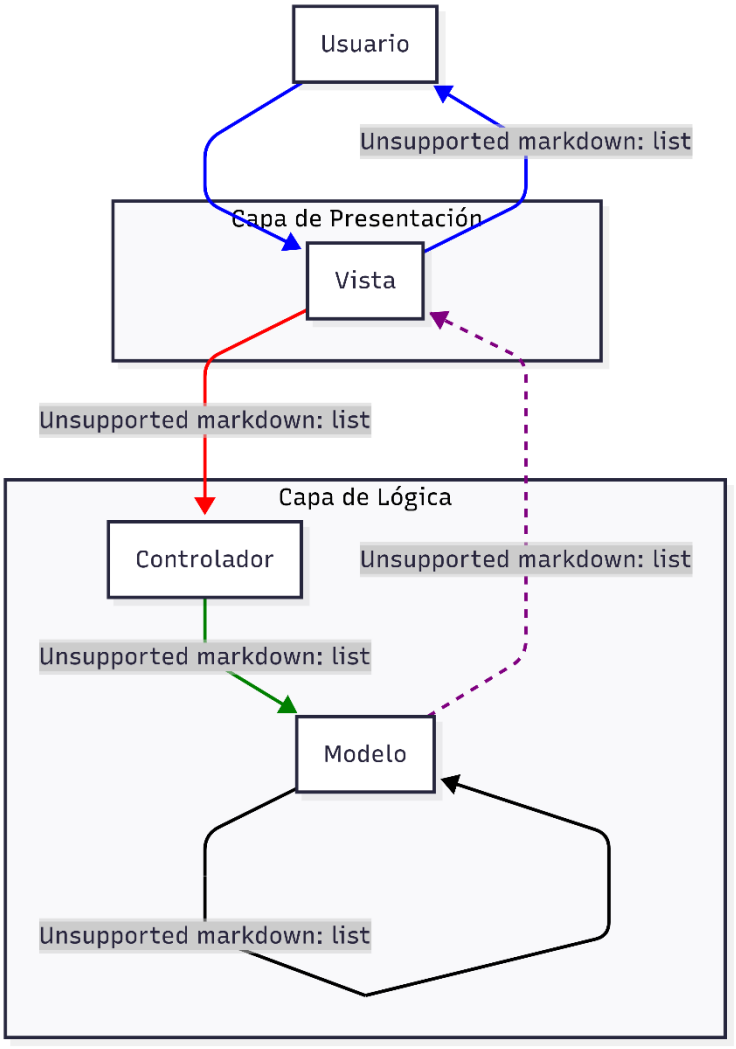

Figura 2 *Diagrama del Patrón Modelo-Vista-Controlador (MVC)*

Diagrama de bloques que ilustra la interacción entre el Usuario, la Vista, el Controlador y el Modelo en el patrón MVC.

Aplicaciones en la vida cotidiana

Los fundamentos teóricos del capítulo anterior son el cimiento, pero el verdadero aprendizaje ocurre cuando vemos esos conceptos en acción. Las Interfaces Gráficas de Usuario no son entidades abstractas; son el puente entre la lógica de un programa y las necesidades de una persona. Están en los teléfonos, en los supermercados, en los bancos y en las oficinas. Este capítulo está diseñado para "desarmar" digitalmente algunas de estas aplicaciones cotidianas, revelando cómo los componentes, contenedores y eventos que estudiamos se orquestan para crear experiencias funcionales en contextos muy familiares para cualquier persona en Colombia.

2.1. El Cajero Automático (ATM): Un Diálogo Guiado por Objetos

El cajero automático es, quizás, uno de los ejemplos más universales y claros de una aplicación de escritorio dirigida por eventos. Su éxito radica en que guía al usuario a través de un proceso complejo (transacciones financieras) de una manera simple y segura, paso a paso. Analicemos su interfaz desde la perspectiva de la POO:

- **La Ventana Principal (JFrame):** Toda la interacción ocurre dentro de una única ventana, un objeto JFrame que actúa como el contenedor principal. Esta ventana no se puede redimensionizar y a menudo se ejecuta en pantalla completa para enfocar la atención del usuario.

- **Componentes de Interacción (JButton, JTextField):**

    - **Pantalla de Bienvenida:** Muestra una imagen y un texto (JLabel).

    - **Ingreso de Clave:** Aunque físicamente es un teclado numérico, en la lógica del software cada tecla es una **fuente de eventos**. Al presionar un número, se genera un evento que un ActionListener captura para añadir el dígito a un campo de contraseña (un JPasswordField que enmascara los caracteres).

    - **Menú de Opciones:** La pantalla que nos pregunta "¿Qué desea hacer?" es un JPanel que contiene varios JButton ("Retiro", "Consulta de Saldo", "Transferencia"). Cada botón es un objeto independiente. Al hacer clic en "Retiro", solo el ActionListener asociado a *ese* botón específico se activa, llevando al usuario a la siguiente pantalla.

- **Flujo Dirigido por Eventos:** El cajero no hace nada hasta que el usuario actúa. La inserción de la tarjeta es el primer evento. El clic en un botón es el siguiente. El programa está en un bucle constante de "esperar-evento, procesar-evento". Esta arquitectura es robusta porque aísla las

funcionalidades. Un error en la lógica de "Transferencia" no afectará la funcionalidad de "Consulta de Saldo", ya que son manejados por listeners y clases distintas, un claro beneficio del encapsulamiento.

Esta interacción guiada y segura es un pilar de la experiencia de usuario (UX) y se logra gracias a la modularidad que nos brinda la POO (Sharp et al., 2019).

2.2. Punto de Venta (POS) en una Tienda: Eficiencia y Velocidad

Pensemos en la caja de una tienda de conveniencia como **Tiendas D1** o una panadería como **Tostao'**. La velocidad es crítica. El cajero no puede permitirse navegar por menús complejos. La GUI de un sistema de Punto de Venta (POS) está optimizada para la eficiencia y la reducción de errores.

- **Diseño Orientado a la Tarea:** La pantalla principal suele estar dividida por JPanels organizados con un GridLayout o GridBagLayout para maximizar el espacio. Un panel muestra los productos más vendidos como botones grandes con imágenes (JButton con ImageIcon). Otro panel contiene un resumen de la venta actual (una JTable). Un tercer panel tiene el teclado numérico y los botones de pago ("Efectivo", "Tarjeta", "Nequi").

- **Historia de Uso Cotidiano:**
    - *8:05 AM en Bogotá. Un cliente pide "un tinto y un pan de bono". La cajera, Ana, no necesita escribir nada. Simplemente presiona el botón con la imagen del café y luego el del pan de bono. Cada clic es un ActionEvent que un ActionListener central captura. Este listener invoca un método en el objeto "Pedido" (Modelo), que añade el producto y actualiza el total. La JTable que muestra la cuenta (Vista) se actualiza inmediatamente. El cliente paga con Nequi; Ana presiona el botón "Nequi", que abre una pequeña ventana de diálogo (JDialog) para confirmar la transacción. Todo el proceso toma menos de 10 segundos.*

- **Análisis Tecnológico:** La clave aquí es la **respuesta inmediata**. La separación del Modelo (la lógica del pedido, los precios) y la Vista (los botones) permite que la interfaz sea extremadamente rápida. Mientras la vista solo se encarga de mostrar botones y capturar clics, el modelo gestiona los cálculos en segundo plano. Este desacoplamiento es fundamental para el rendimiento en aplicaciones transaccionales de alto volumen.

2.3. Aplicaciones de Oficina: Gestión de Citas en una EPS

Las aplicaciones de escritorio siguen siendo vitales en entornos corporativos y de gestión, como en el sistema de agendamiento de una EPS en Colombia. Estas GUIs

manejan una mayor densidad de información y se enfocan en la organización y visualización de datos.

- **Componentes Complejos:** Estas interfaces utilizan componentes más avanzados. Por ejemplo, un calendario para seleccionar fechas (como el componente JCalendar de una librería externa), tablas (JTable) para mostrar listas de pacientes con barras de desplazamiento (JScrollPane), y formularios complejos con menús desplegables (JComboBox) para seleccionar especialistas o tipos de consulta.

- **Testimonio Ficticio (Recepcionista en una EPS de Cali):**

"Antes usábamos un cuaderno. Era un caos. Si un paciente cancelaba, tocaba tachar y buscar el hueco. Ahora, con el software, es diferente. Tengo la semana entera en la pantalla del calendario. Cuando alguien llama, busco al paciente por cédula en un campo de texto. Su historial aparece al instante. Hago clic en un día libre, selecciono la hora en un menú y listo, la cita queda agendada. El sistema me muestra alertas si hay un cruce de horarios. Me ahorra al menos una hora al día y hemos reducido los errores de agendamiento en un 90%. Para mí, esa pantalla es mi herramienta de trabajo principal."

- **Vinculación con la Lógica de Negocio:** Este ejemplo ilustra perfectamente la conexión entre la **Vista** (el formulario de citas) y el **Modelo** (las reglas de negocio: un médico no puede tener dos citas a la vez, una cita debe durar 20 minutos, etc.). Cuando el recepcionista presiona "Guardar Cita" (JButton), el Controlador toma los datos del formulario, los valida contra las reglas del Modelo y, si todo es correcto, los guarda en la base de datos. Si hay un error (como un cruce de horarios), el Modelo lo notifica al Controlador, que a su vez le pide a la Vista que muestre un mensaje de error (JOptionPane). Este flujo, basado en el patrón MVC, es lo que hace que la aplicación sea robusta y fiable (Domínguez Mínguez, 2022).

Casos de estudio

Mientras que las aplicaciones de la vida real nos muestran *qué* hacen las interfaces gráficas, un caso de estudio profesional nos revela *cómo* se construyen desde una perspectiva de ingeniería y gestión. Aquí es donde el rol del tecnólogo brilla, pues no solo se trata de escribir código, sino de tomar decisiones estratégicas, justificar el uso de una tecnología sobre otra y estructurar la solución de una manera que sea eficiente, mantenible y escalable. Este capítulo te sumergirá en un proyecto ficticio pero verosímil, desde la definición del problema hasta la medición del impacto, aplicando el rigor metodológico que demanda la industria.

3.1. Título del Caso: "SIGA-CUN: Sistema Integrado de Gestión Académica de Escritorio"

- **Cliente:** Una sede regional de la CUN con infraestructura de red intermitente.
- **Contexto:** La sede gestiona sus procesos de matrícula, inscripción de asignaturas y consulta de notas de forma manual, utilizando hojas de cálculo y documentos físicos. Esto genera inconsistencias, pérdida de datos y una gran sobrecarga administrativa al inicio de cada semestre.
- **Necesidad:** Requieren una aplicación de escritorio centralizada que pueda operar de manera local en la red de la oficina, sin depender de una conexión a internet constante, garantizando la seguridad y la rapidez en la consulta y gestión de la información estudiantil.

3.2. El Desafío: Requerimientos y Restricciones Clave

El equipo de desarrollo recibió un conjunto de requerimientos claros:

1. **Funcionalidad Principal:** El sistema debe permitir realizar CRUD (Crear, Leer, Actualizar, Borrar) para las entidades de Estudiantes, Docentes y Asignaturas. El módulo más crítico es el de matriculación, que debe permitir inscribir un estudiante en múltiples asignaturas, validando cupos y pre-requisitos.
2. **Operación Offline:** La principal restricción es que la aplicación debe ser 100% funcional dentro de la red local de la sede. La sincronización con un servidor central se podría hacer en lotes al final del día, pero la operación diaria no puede depender de internet.
3. **Seguridad:** Los datos de los estudiantes son sensibles. El acceso al sistema debe estar controlado por roles (Administrativo, Coordinador) y la base de datos debe residir en un servidor seguro dentro de la misma sede.
4. **Usabilidad:** El personal administrativo tiene distintos niveles de habilidad tecnológica. La interfaz debe ser extremadamente intuitiva, limpia y requerir una capacitación mínima.

3.3. La Solución Aplicada (Enfoque de Tecnólogo)

Tras analizar los requerimientos, el equipo de desarrollo tomó las siguientes decisiones estratégicas:

- **Selección de Tecnología:** Se optó por desarrollar una **aplicación de escritorio con Java y la librería Swing**.
  - **Justificación vs. Aplicación Web:** Aunque una aplicación web es más moderna, no cumplía el requisito de operación offline. Una aplicación de escritorio garantiza que la sede pueda seguir matriculando incluso si se cae el servicio de internet. Además, al ejecutarse en la JVM (Java Virtual Machine), la aplicación es multiplataforma por naturaleza, pudiendo correr en los equipos Windows o Linux de la institución.
- **Arquitectura de Software: Patrón Modelo-Vista-Controlador (MVC)**
  - Para garantizar la separación de responsabilidades y la mantenibilidad a futuro, se implementó rigurosamente el patrón MVC. Esta decisión es crucial desde el punto de vista tecnológico porque permite que el sistema evolucione. Por ejemplo, si en el futuro se decide crear una versión web, el **Modelo** (toda la lógica de negocio y acceso a datos) podría reutilizarse casi en su totalidad (Pressman & Maxim, 2021).

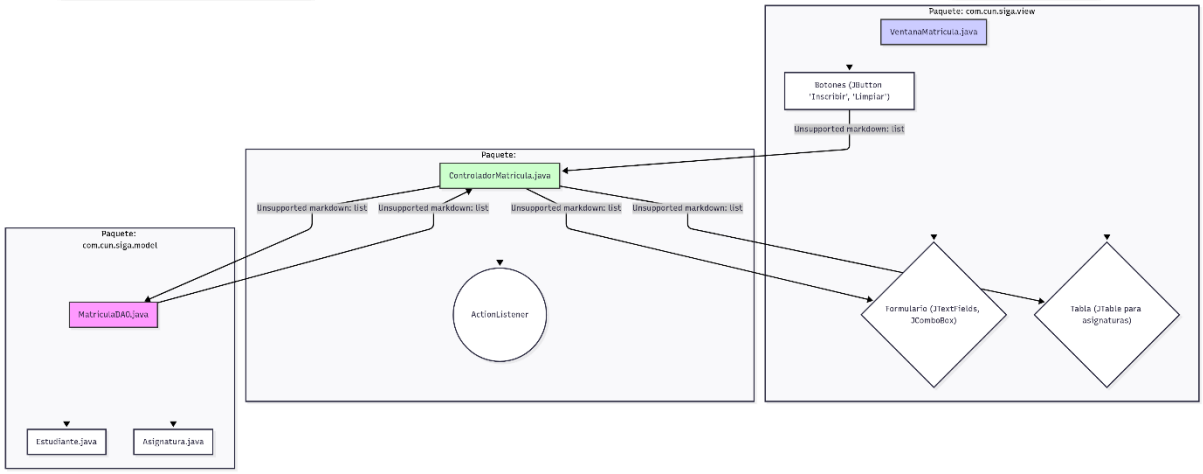

Figura 3 *Arquitectura MVC aplicada al Módulo de Matrícula del SIGA-CUN*

Diagrama de arquitectura MVC mostrando cómo la VentanaMatricula (Vista) se comunica con el ControladorMatricula, que a su vez manipula los objetos del Modelo como Estudiante y MatriculaDAO.

- **Implementación Orientada a Objetos:**
  - **Modelo:** Se crearon clases Estudiante, Docente y Asignatura que reflejaban directamente las entidades del mundo real (abstracción). Cada clase encapsulaba sus propios datos y métodos (ej:

estudiante.getNombre()). Se utilizó una clase MatriculaDAO (Data Access Object) para encapsular toda la lógica de interacción con la base de datos, haciendo el resto del código independiente de si se usaba MySQL, PostgreSQL u otra.

- **Vista:** Cada pantalla principal (ej: "Gestión de Estudiantes", "Proceso de Matrícula") se implementó como una clase que heredaba de JFrame o JPanel. Esto permitió reutilizar componentes comunes, como una barra de menú o un pie de página.

- **Controlador:** Se creó un Controlador específico para cada vista principal. El ControladorMatricula, por ejemplo, implementaba la interfaz ActionListener y se registraba con todos los botones de la VentanaMatricula. Su método actionPerformed contenía un switch o if-else para determinar qué botón se presionó y qué acción tomar.

3.4. Impacto y Resultados Cuantificables

Seis meses después de la implementación del **SIGA-CUN**, se realizó una evaluación de impacto con los siguientes resultados clave:

- **Reducción de Tiempo:** El tiempo promedio para completar una matrícula por estudiante, incluyendo la inscripción de todas sus asignaturas y la generación del recibo, **se redujo en un 40%**, pasando de 15 minutos a un promedio de 9 minutos.
- **Disminución de Errores:** Los errores de inscripción (cruces de horarios, inscripción en asignaturas sin cumplir pre-requisitos) se **eliminaron en un 98%**, ya que todas estas validaciones eran realizadas automáticamente por el Modelo antes de confirmar la operación.
- **Satisfacción del Personal:** Una encuesta interna mostró un **aumento del 75% en la satisfacción del personal administrativo** con sus herramientas de trabajo. La percepción de control, la reducción de estrés durante la temporada de matrículas y la fiabilidad de los datos fueron los puntos más valorados.

El éxito del proyecto validó la decisión de usar una arquitectura robusta y desacoplada, demostrando que una buena planificación y un diseño de software sólido son tan importantes como el código mismo (Larman, 2012).

Herramientas y técnicas

Saber qué construir es la mitad del trabajo; saber cómo construirlo eficientemente es la otra. Un desarrollador tecnólogo no solo domina los conceptos, sino también las herramientas que le permiten materializar sus diseños de forma rápida y profesional. Este capítulo se centra en el "cómo". Exploraremos el entorno de desarrollo integrado (IDE) NetBeans y su potente diseñador visual de interfaces, que nos permite prototipar y construir GUIs de manera intuitiva. Luego, pondremos en práctica lo aprendido con ejercicios guiados que te llevarán desde la creación de tu primera ventana hasta la personalización de su apariencia, reforzando siempre las buenas prácticas que separan a un aficionado de un profesional.

4.1. El Entorno de Desarrollo: NetBeans y su Diseñador de GUI "Matisse"

Apache NetBeans es un IDE muy popular para el desarrollo en Java, y una de sus características más aclamadas es su editor de GUI, conocido internamente como "Matisse". Este editor es una herramienta de **Diseño Visual (WYSIWYG - What You See Is What You Get)**, que transforma la tarea de diseñar una ventana en un proceso similar a dibujar en un programa de diseño gráfico.

- **La Paleta de Componentes:** A un lado de la pantalla, NetBeans presenta una "Paleta" con todos los componentes de Swing (y JavaFX si está configurado). ¿Necesitas un botón? Simplemente lo arrastras desde la paleta y lo sueltas sobre tu JFrame o JPanel.
- **El Inspector de Propiedades:** Al seleccionar cualquier componente en el diseñador (un JButton, por ejemplo), el "Inspector" muestra todas sus propiedades: el texto (text), el color de fondo (background), el tamaño (preferredSize), etc. Puedes modificar estas propiedades directamente en el inspector, y el diseñador visual te mostrará el cambio en tiempo real.
- **Generación Automática de Código:** La verdadera magia de Matisse es que, mientras tú diseñas visualmente, NetBeans escribe el código Java correspondiente por ti. Genera todo el código de inicialización, posicionamiento (usando GroupLayout, su layout por defecto) y configuración de los componentes en un método llamado initComponents(). Este código está protegido (es de color gris y no editable) para evitar modificaciones accidentales que rompan el diseñador visual. Tú solo debes preocuparte por escribir el código que maneja los eventos.

Esta herramienta acelera drásticamente el desarrollo, pero es crucial entender el código que genera para poder depurar y optimizar la aplicación eficazmente (Farrell, 2019).

4.2. Ejercicio Práctico 1: Creando una Calculadora Básica con Swing

Este ejercicio es un rito de iniciación en el desarrollo de GUIs. Nos permitirá aplicar los conceptos de componentes, contenedores y, sobre todo, la gestión de eventos.

Guía Paso a Paso:

1. **Crear el Proyecto:** En NetBeans, crea un nuevo "Java with Ant" -> "Java Application".
2. **Crear el JFrame:** Haz clic derecho en tu paquete de fuentes -> "New" -> "JFrame Form". Nómbralo CalculadoraUI. NetBeans abrirá el diseñador visual.
3. **Diseñar la Interfaz:**
   - Arrastra un JTextField a la parte superior del JFrame. En sus propiedades, llámalo txtDisplay y borra su texto inicial.
   - Arrastra un JPanel debajo del campo de texto. En este panel irán los botones.
   - Configura el Layout Manager del JPanel a GridLayout. Un GridLayout de 4x4 es ideal para una calculadora estándar.
   - Arrastra 16 JButton dentro del JPanel. Edita sus propiedades de text para que muestren los números (0-9), los operadores (+, -, *, /), el igual (=) y un botón de limpiar (C).
4. **Implementar la Lógica (El ActionListener):**
   - Haz doble clic en el botón "1". NetBeans te llevará automáticamente al código y creará un método de evento para ti (jButton1ActionPerformed).
   - Dentro de este método, escribe el código para que el número "1" se añada al txtDisplay:

```
String textoActual = this.txtDisplay.getText();
this.txtDisplay.setText(textoActual + "1");
```

   - Repite este proceso para todos los botones de números y operadores. Para el botón "=", deberás implementar la lógica que evalúe la expresión matemática. Para un tecnólogo, un desafío interesante sería usar una librería como exp4j para evaluar la cadena de texto en lugar de implementar un parser manual (Ortega, 2020).

4.3. Ejercicio Práctico 2: Personalizando la Apariencia con UIManager

Una aplicación Swing por defecto puede verse algo anticuada. La clase UIManager nos permite cambiar el **"Look and Feel" (L&F)** de toda la aplicación con unas pocas líneas de código, para darle una apariencia más moderna o nativa del sistema operativo donde se ejecuta.

- **¿Cómo se implementa?** Justo al inicio de tu método main, antes de crear ninguna ventana, puedes insertar el siguiente código para intentar establecer el L&F del sistema operativo:

```
public static void main(String args[]) {
    try {
        // Establece el Look and Feel del sistema operativo actual (Windows, macOS,
Linux GTK+)
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (ClassNotFoundException | InstantiationException |
IllegalAccessException | UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(CalculadoraUI.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);
    }

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new CalculadoraUI().setVisible(true);
        }
    });
}
```

Este simple cambio hace que la aplicación se sienta más integrada y profesional, una técnica de optimización de la experiencia de usuario muy valorada (Schildt, 2019).

4.4. Buenas Prácticas: Separando la Lógica del Diseño

El error más común al usar diseñadores visuales es poner toda la lógica del programa dentro de los métodos de evento de la ventana. Esto viola el principio de separación de responsabilidades y crea "código espagueti".

- **La Regla de Oro:** La clase de la Vista (tu CalculadoraUI.java) solo debe contener código relacionado con la apariencia y la manipulación de los componentes. **Nunca** debe contener la lógica de negocio.
- **Aplicación Práctica (Refactorizando la Calculadora):**
    1. Crea una nueva clase Java llamada CalculadoraLogica.java (el **Modelo**).
    2. Esta clase tendrá métodos como agregarDigito(String digito), agregarOperador(String op) y calcularResultado().
    3. En tu CalculadoraUI.java (la **Vista**), crea una instancia de CalculadoraLogica.

4. Ahora, el actionPerformed del botón "1" no manipulará directamente el JTextField. En su lugar, llamará a miCalculadoraLogica.agregarDigito("1"); y luego actualizará el JTextField con el estado actual del modelo.

Esta separación, aunque requiere un poco más de código inicial, hace que el programa sea infinitamente más fácil de probar, depurar y extender en el futuro.

Conclusión

A lo largo de este recorrido, hemos transitado desde los fundamentos teóricos de la Programación Dirigida por Eventos hasta la construcción práctica de una aplicación con herramientas profesionales. Hemos desmitificado la "magia" detrás de las ventanas y botones, descubriendo que cada elemento de una interfaz gráfica es, en esencia, un objeto que colabora con otros dentro de un ecosistema bien definido. La Programación Orientada a Objetos se ha revelado no como un simple estilo de codificación, sino como el paradigma ideal para gestionar la complejidad inherente a las aplicaciones interactivas. Principios como el **encapsulamiento** nos permitieron crear componentes autónomos y seguros, la **herencia** facilitó la construcción de elementos especializados a partir de otros más generales, y el **polimorfismo**, a través de los listeners, nos dio la flexibilidad para que diferentes objetos respondan a un mismo tipo de evento de maneras únicas.

Hemos visto cómo esta teoría cobra vida en aplicaciones tan cotidianas como un cajero automático o el sistema de una tienda, demostrando que el software que usamos a diario se sostiene sobre estos pilares conceptuales. El caso de estudio del "SIGA-CUN" nos enseñó que las decisiones de un tecnólogo van más allá del código: implican analizar requerimientos, justificar tecnologías y, fundamentalmente, diseñar arquitecturas limpias como el **Modelo-Vista-Controlador (MVC)**, que garantizan que el software pueda crecer y adaptarse sin colapsar bajo su propio peso.

El verdadero aprendizaje, sin embargo, comienza ahora. La invitación es a que no te detengas en la calculadora de ejemplo. Mira a tu alrededor: la interfaz del microondas, la app de tu servicio de streaming, el software de la biblioteca. Intenta descomponerlos mentalmente. ¿Qué componentes usan? ¿Cómo manejarán los eventos? ¿Cómo estarán separados su Modelo y su Vista? Al aplicar este pensamiento crítico, transformarás tu manera de ver el software, pasando de ser un simple usuario a ser un arquitecto de experiencias. El desarrollo de interfaces gráficas es un campo que fusiona la lógica rigurosa con la empatía hacia el usuario, y dominarlo te abrirá innumerables puertas profesionales.
Para continuar tu viaje, te dejamos con dos preguntas abiertas que esperamos despierten tu curiosidad y te impulsen a investigar más allá de estas páginas:

1. En un mundo que se mueve cada vez más hacia las aplicaciones móviles y web, ¿qué nichos o industrias en el contexto colombiano consideras que seguirán dependiendo críticamente de las aplicaciones de escritorio robustas en los próximos cinco años y por qué?
2. Más allá de la funcionalidad, ¿cómo pueden los principios de diseño de experiencia de usuario (UX), como la simplicidad, la retroalimentación y la consistencia, ser implementados en el código de una aplicación Java Swing para crear no solo un programa que funcione, sino uno que sea un placer utilizar?

Referencias.

Deitel, P. J., & Deitel, H. M. (2012). *Java: How to Program*. Prentice Hall.

Domínguez Mínguez, T. (2022). *Tkinter. Desarrollo de interfaces gráficas en Python*. RA-MA S.A. Editorial y Publicaciones.

Farrell, J. (2019). *Java Programming*. Cengage Learning.

Joyanes Aguilar, L. (2020). *Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos*. McGraw-Hill.

Larman, C. (2012). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall.

Ortega, J. (2020). *Java para estudiantes*. IT Campus Academy.

Pressman, R. S., & Maxim, B. R. (2021). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.

Schildt, H. (2019). *Java: The Complete Reference*. McGraw-Hill Education.

Sharp, H., Preece, J., & Rogers, Y. (2019). *Interaction Design: Beyond Human-Computer Interaction*. John Wiley & Sons.

Sommerville, I. (2011). *Software Engineering*. Pearson.

Stallings, W. (2005). *Operating Systems: Internals and Design Principles*. Pearson/Prentice Hall.